Decisões de Arquitetura

Equipe ManaTrade

28/09/2026

3.1 Modelagem de Dados: PostgreSQL + MongoDB (persistência poliglota)

Contexto: o sistema precisa persistir dados transacionais (usuários, lojas, anúncios, consignações, transações, avaliações) que exigem integridade referencial e atomicidade, e também um catálogo de cartas com atributos variáveis por edição, histórico de preços e comentários, cuja estrutura muda com frequência e que funciona como cache com expiração da API Scryfall.

Alternativas consideradas:

- Usar apenas PostgreSQL para tudo, modelando o catálogo de cartas em tabelas relacionais rígidas.
- Usar apenas MongoDB para tudo, incluindo os dados transacionais.
- Usar Firebase Firestore como back-end único, conforme sugestão da disciplina.

Escolha: adotar modelagem híbrida (persistência poliglota): PostgreSQL via Sequelize para entidades transacionais com integridade referencial forte (`Usuario`, `Loja`, `Anuncio`, `Consignacao`, `Transacao`, `Avaliacao`); MongoDB via Mongoose para o catálogo de cartas, histórico de preços e logs (`Carta`, `LogPreco`, `Comentario`), com TTL de 7 dias para expirar o cache.

Consequências:

- Cada banco é usado para o que faz melhor: garantia ACID nas transações financeiras e flexibilidade de esquema no catálogo.
- O cache de cartas no MongoDB reduz chamadas à API Scryfall e melhora o tempo de resposta.
- Aumenta a complexidade operacional: dois bancos e dois ORMs (Sequelize e Mongoose) para manter.
- Exige disciplina para manter consistência quando um anúncio (PostgreSQL) referencia dados do catálogo (MongoDB).

3.2 Autenticação: JWT + Refresh Token

Contexto: o aplicativo precisa de sessão persistente no mobile e proteção de telas privadas, sem depender de um serviço de terceiros e mantendo controle total do back-end

sobre o processo de autenticação.

Alternativas consideradas:

- Firebase Authentication, conforme sugestão da disciplina.
- Sessões tradicionais baseadas em cookie, com estado mantido no servidor.
- Login social (Google/Apple) como único método de autenticação.

Escolha: JWT de curta duração (15 minutos) combinado a um Refresh Token de longa duração (7 dias); o refresh token é armazenado de forma segura no SecureStore do dispositivo, as rotas privadas validam o JWT a cada requisição, e o token é renovado automaticamente ao expirar.

Consequências:

- A API se mantém stateless, facilitando a escalabilidade horizontal do back-end.
- A equipe não fica dependente de um serviço externo (Firebase) nem dos limites do seu plano gratuito.
- Em contrapartida, a equipe assume a responsabilidade por implementar e manter toda a lógica de emissão, validação, expiração e revogação dos tokens.

3.3 Funcionalidade de IA Generativa: Google Gemini (ManaScan)

Contexto: a disciplina exige uma funcionalidade de IA generativa genuinamente útil dentro do fluxo do usuário. O problema real identificado é que avaliar a edição, condição e preço justo de uma carta manualmente é trabalhoso e subjetivo.

Alternativas consideradas:

- Não incluir IA e exigir que o usuário preencha manualmente todos os dados da carta — descartada por não atender ao requisito da disciplina e por piorar a experiência do usuário.
- Utilizar a API de visão computacional da OpenAI (GPT-4o).
- Utilizar um modelo de reconhecimento de imagem embarcado no dispositivo (ex.: TensorFlow Lite).

Escolha: Google Gemini (`gemini-1.5-flash`), via Google AI Studio, com suporte a visão computacional e tier gratuito com limites generosos para desenvolvimento. A IA analisa a foto da carta e sugere nome, edição, condição e preço, cruzando o histórico de preços do aplicativo com os dados da Scryfall.

Consequências:

- O ManaScan passa a ser o diferencial central do aplicativo: sem ele, a experiência do usuário fica claramente pior, atendendo ao critério de utilidade genuína exigido pela disciplina.
- O tier gratuito reduz o custo durante o desenvolvimento e a apresentação do projeto.
- Cria uma dependência de um serviço externo sujeito a limites de uso e indisponibilidade, exigindo tratamento de falha com preenchimento manual como alternativa.

- Exige cuidado redobrado para que a chave de API nunca seja versionada no repositório.

3.4 Back-end: Node.js + Express em padrão MVC

Contexto: a equipe precisa de uma API REST consumida pelo aplicativo mobile e de um painel administrativo para as lojas, mantendo o código organizado e de fácil manutenção entre os integrantes do grupo.

Alternativas consideradas:

- Back-end serverless com Firebase (Firestore + Cloud Functions), conforme sugestão da disciplina.
- Um framework mais opinativo, como NestJS.
- Painel web renderizado em um framework front-end separado (React ou Vue), consumindo a mesma API REST.

Escolha: back-end próprio em Node.js e Express, organizado em padrão MVC (`config/`, `controllers/`, `models/`, `routers/`, `views/`), com o painel web das lojas renderizado via Express-Handlebars no mesmo servidor da API REST.

Consequências:

- Todo o projeto usa a mesma linguagem (JavaScript) no mobile e no back-end.
- A separação clara de responsabilidades do padrão MVC facilita a divisão de tarefas entre os integrantes do grupo.
- A equipe mantém controle total do back-end, sem depender dos limites do plano gratuito do Firebase.
- Em contrapartida, a equipe assume a responsabilidade por hospedagem, escalabilidade e segurança do próprio servidor, sem suporte da disciplina para essa parte, conforme alertado no enunciado do trabalho.